

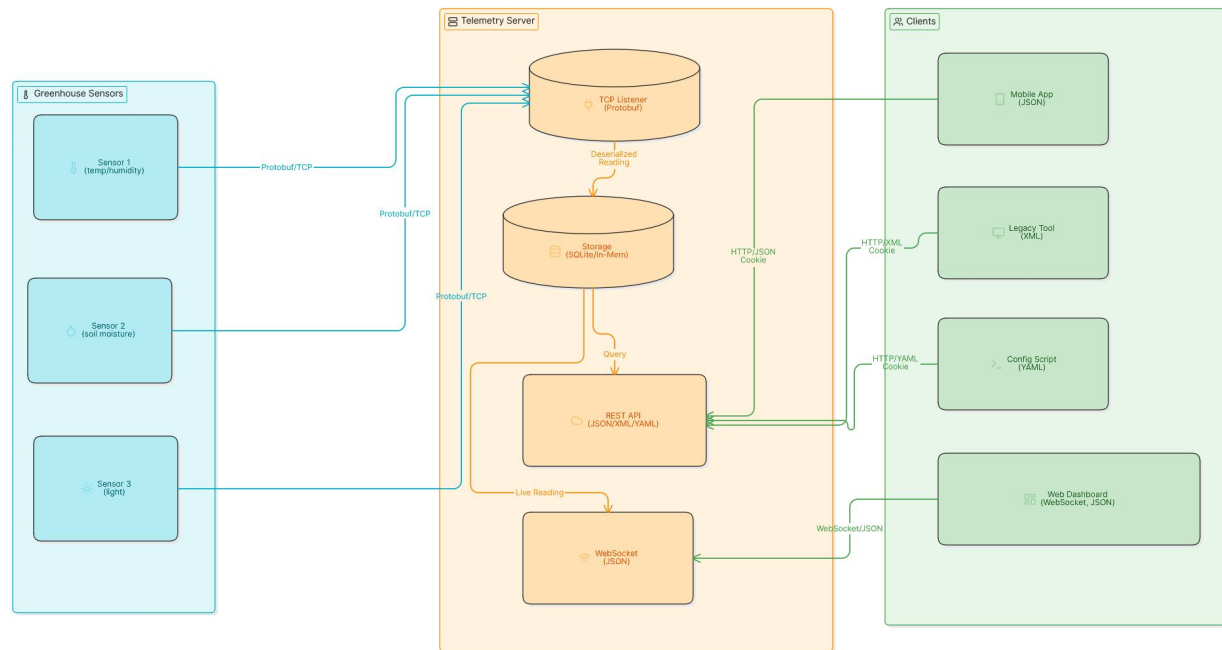
NET322: Network Programming & Applications Development

Practical Test 2 — Assignment 2: Planning & Architecture

IoT Telemetry Pipeline & Dashboard

1. System Architecture

System Architecture: Greenhouse Telemetry Pipeline



The system is split into three layers: sensors on the left, a central server in the middle, and clients on the right. Data always flows left to right — sensors push readings to the server, and the server delivers them to clients.

Sensor Layer

Each greenhouse has one or more sensors measuring temperature, humidity, soil moisture, or light. A configuration file tells each sensor its ID, what it measures, and how often to send a reading. Sensors connect to the server over a persistent TCP connection and keep sending readings on that fixed schedule until they are shut down or disconnected.

Telemetry Server

The server is the central hub. It has four responsibilities:

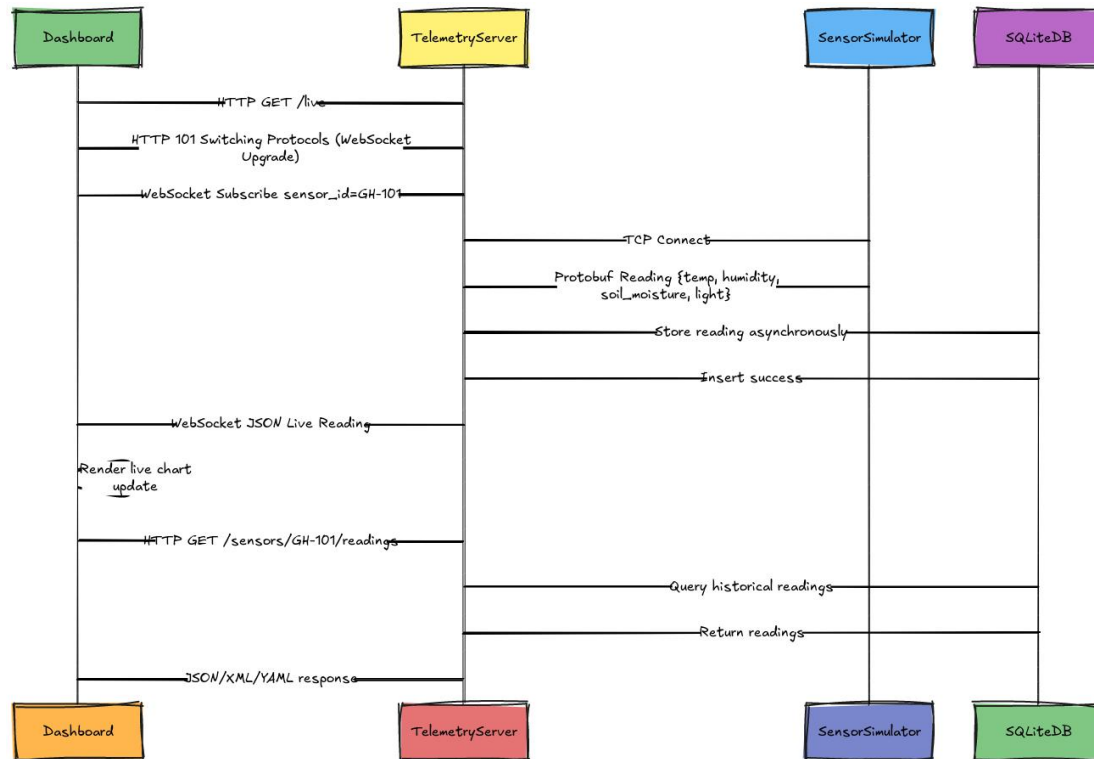
- Accept incoming sensor connections and receive their readings.
- Store every reading so it can be retrieved later.
- Answer queries from REST API clients about historical data.
- Push live readings to any dashboard that is currently connected.

All of this runs in a single process with no threads — the server handles many sensors and clients at the same time using asynchronous programming (AsyncIO).

Client Layer

Three types of client query historical data through the REST API: a mobile app that prefers JSON, a legacy desktop tool that reads XML, and a configuration script that uses YAML. The server detects which format each client wants from the request header and responds accordingly. A fourth client — the web dashboard — connects over WebSocket to receive readings in real time as they arrive.

2. Sequence Diagram



The sequence diagram shows what happens from the moment a sensor sends a reading to the moment a dashboard displays it, plus how REST queries and disconnections are handled.

Step 1 — Dashboard Connects (WebSocket Handshake)

Before any data flows, the dashboard browser opens a connection to the server at the /live address. It sends a standard HTTP request asking to upgrade to a WebSocket connection. The server agrees and the connection switches to a two-way channel. The dashboard can then tell the server which sensors it wants to watch.

Step 2 — Sensor Sends a Reading

On a fixed schedule, the sensor packages its latest measurement into a compact binary message (protobuf) and sends it to the server over its TCP connection. The server receives the message, unpacks it, and reads the sensor ID, measurement type, value, and timestamp.

Step 3 — Server Stores and Broadcasts

The server does two things with each incoming reading. First, it saves it to storage so it can be retrieved later by the REST API. Second, it immediately forwards the reading as a JSON message to every

dashboard client that is subscribed to that sensor. Both of these happen in the same breath — storage and broadcast run back to back without any delay.

Step 4 — Dashboard Displays the Reading

The dashboard receives the JSON message and plots the new value on its live chart. This happens within milliseconds of the sensor sending the reading.

Step 5 — REST Client Queries History

A mobile app, legacy tool, or config script sends an HTTP GET request for a sensor's readings over a time range. It includes an Accept header to say which format it wants back. The server looks up the matching readings in storage, serialises them into the requested format, and returns them along with a session cookie. On the next request, the client sends that cookie back so the server recognises it.

Step 6 — Sensor Disconnects

If a sensor drops off the network, the server notices the broken connection, logs it, and stops waiting for that sensor. The sensor automatically tries to reconnect after a short wait, backing off gradually if the server is not yet reachable.

Step 7 — Malformed Message Arrives

If a sensor accidentally sends a garbled or corrupted message, the server logs a warning and moves on. The connection is kept open and the next valid message is processed normally.

3. Data Model

The system stores two kinds of data: information about each registered sensor, and the individual readings that sensors produce.

Sensor Record

Each sensor has a unique ID, a type (temperature, humidity, soil moisture, or light), a reporting interval in seconds, and an optional location label. The server records the time the sensor was first registered.

Field	Type	Description
sensor_id	Text	Unique identifier, e.g. GH-001
type	Text	One of: temperature, humidity, soil_moisture, light
interval_seconds	Number	How often the sensor reports, in seconds
location	Text	Optional label, e.g. Greenhouse 1 — North
registered_at	Timestamp	When the sensor was first seen by the server

Reading Record

Each reading belongs to a sensor and captures a single measurement at a point in time.

Field	Type	Description
id	Integer	Automatically assigned, unique per reading
sensor_id	Text	Links the reading back to its sensor
value	Decimal	The measured value in the sensor's native unit
timestamp	Integer	When the reading was taken, as a Unix timestamp (UTC)

Wire Format — Protobuf Schema

Messages sent over TCP between sensors and the server use Protobuf, a compact binary format. The schema defines three things: the list of sensor types, the structure of a single reading, and the structure of a sensor registration message. A reading carries the sensor ID, type, value, and timestamp. A registration message also includes the reporting interval and location label. Each message is preceded by a 4-byte number telling the server how many bytes to read.

```
syntax = "proto3";
package telemetry;
// Enumerates sensor measurement types.
enum SensorType {
    UNKNOWN = 0;
    TEMPERATURE = 1; // degrees Celsius
    HUMIDITY = 2; // percentage 0-100
    SOIL_MOISTURE = 3; // percentage 0-100
    LIGHT = 4; // lux
}

// A single sensor reading pushed by a sensor simulator.
message Reading {
    string sensor_id = 1; // e.g. "GH-001"
    SensorType type = 2; // measurement category
    double value = 3; // measured value
    int64 timestamp = 4; // Unix epoch seconds (UTC)
}

// Sent by a sensor when it first connects, to register itself.
message SensorRegistration {
    string sensor_id = 1;
    SensorType type = 2;
    double interval_seconds = 3; // reporting cadence
    string location = 4; // optional: e.g. "Greenhouse 3 - North"
}
```

4. REST Endpoint Specification

The REST API has four endpoints. Every response includes a session cookie so the server can recognise returning clients. The format of the response body depends on the Accept header sent by the client.

List All Sensors

Method & path	GET /sensors
What it does	Returns a list of all registered sensors
Request body	None
Success response	200 OK — array of sensor objects in the requested format
Error response	500 if the storage layer fails
Example	Returns sensor ID, type, interval, location, and registration time for each sensor

Get Historical Readings

Method & path	GET /sensors/{id}/readings
What it does	Returns all readings for a specific sensor, optionally filtered by time range
Query parameters	from — start time as a Unix timestamp (optional); to — end time (optional)
Request body	None
Success response	200 OK — array of reading objects
Error responses	404 if the sensor does not exist; 400 if the time parameters are invalid

Register a Sensor

Method & path	POST /sensors
What it does	Adds a new sensor to the system
Request body	Sensor ID, type, interval, and optional location — in JSON or YAML
Success response	201 Created — the new sensor record; Location header points to the sensor URL
Error responses	409 if the sensor ID is already taken; 400 if a required field is missing

Remove a Sensor

Method & path	DELETE /sensors/{id}
--------------------------	----------------------

What it does	Removes a sensor and all its readings from storage
Request body	None
Success response	204 No Content
Error response	404 if the sensor does not exist

Content Negotiation and Session Cookie

Every response carries a Set-Cookie header with a session identifier. If the client already has a cookie, the server reads it and keeps the session going. The format of the response body is decided by the Accept header:

Accept header value	Response format
application/json	JSON
application/xml	XML
application/yaml	YAML
Not sent / unrecognised	JSON (default)
None of the above	406 Not Acceptable

HTTP Error Codes

The REST API uses standard HTTP status codes to communicate errors clearly.

Situation	Status code returned
Everything worked	200 OK (or 201 Created for new sensors)
Sensor or resource not found	404 Not Found
Invalid request parameter	400 Bad Request
Sensor ID already registered	409 Conflict
Client asked for an unsupported format	406 Not Acceptable
Storage temporarily unavailable	503 Service Unavailable
Unexpected internal problem	500 Internal Server Error

Working Examples

The following examples show real HTTP request/response pairs for each endpoint, demonstrating content negotiation and session cookie behaviour.

Example 1 — GET historical readings (JSON client)

Request (mobile app, prefers JSON):

```
GET /sensors/GH-001/readings?from=1746700800&to=1746787200 HTTP/1.1
Host: localhost:8080
Accept: application/json
Cookie: session_id=abc123xyz
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/json
Set-Cookie: session_id=abc123xyz; HttpOnly; Path=/

{
  "sensor_id": "GH-001",
  "readings": [
    { "id": 1, "value": 24.3, "timestamp": 1746700812, "unit": "°C" },
    { "id": 2, "value": 24.7, "timestamp": 1746700817, "unit": "°C" }
  ]
}
```

5. YAML Configuration Schema

Each sensor simulator reads a single YAML file when it starts. The file tells it where the server is and describes every sensor it should run.

Fields

```
server:
  host: <string> # IP or hostname of the telemetry server
  port: <integer> # TCP ingest port (default: 9000)
sensors:
  - id: <string> # Unique sensor identifier
    type: <enum> # temperature | humidity | soil_moisture | light
    interval_seconds: <num> # Reporting cadence in seconds (float > 0)
    location: <string> # Optional: human-readable location
    # Optional per-type parameters:
    min_value: <num> # Lower bound for generated readings
    max_value: <num> # Upper bound for generated readings
```

Example Configuration File

The following is a sample configuration with four sensors representing a typical greenhouse setup. The server is running locally on port 9000.

```
server:
  host: 127.0.0.1
  port: 9000
sensors:
  - id: GH-001
    type: temperature
    interval_seconds: 5
    location: Greenhouse 1 - North
    min_value: 15.0
    max_value: 35.0
```

```

- id: GH-002
  type: humidity
  interval_seconds: 10
  location: Greenhouse 2 - South
  min_value: 40.0
  max_value: 90.0

- id: GH-003
  type: soil_moisture
  interval_seconds: 30
  location: Farm A - Field 3
  min_value: 20.0
  max_value: 80.0

- id: GH-004
  type: light
  interval_seconds: 15
  location: Greenhouse 1 - South
  min_value: 0.0
  max_value: 100000.0

```

6. Concurrency Model

The entire server runs on a single thread using AsyncIO — Python's built-in system for handling many tasks at once without creating new threads. Instead of blocking to wait for data, each task pauses and lets others run, then resumes when its data arrives. This is well suited to network servers where most of the time is spent waiting for messages, not doing computation.

Tasks and What They Do

Task	What it does	How it communicates
Main startup	Starts the TCP server and the HTTP server, then waits indefinitely.	Creates all other tasks
Sensor handler	One runs per connected sensor. Waits for a message, processes it, then waits again. Ends when the sensor disconnects.	Calls storage and broadcaster directly
REST request handler	One runs per HTTP request. Reads the request, queries storage, serialises the result, and replies.	Calls storage; reads/sets session cookie
WebSocket handler	One runs per connected dashboard. Waits for subscription messages and stays open until the client disconnects.	Registers itself with the broadcaster
Broadcaster	Called by the sensor handler after each reading. Sends the reading to all matching WebSocket clients at the same time.	Calls each client's send function concurrently

What Happens When a Client Is Too Slow

If a dashboard client is receiving messages more slowly than the server is sending them, each send attempt is given a 2-second time limit. If the client does not accept the message within that time, it is disconnected and removed from the list. This prevents one slow client from holding up the rest of the system.

Storage Choice — SQLite

SQLite was chosen over a pure in-memory store because readings need to survive a server restart. SQLite is a file-based database that requires no separate server process, making it easy to set up and appropriate for a prototype of this scale. Write-ahead logging is enabled so reads and writes can overlap without locking each other out.

7. Failure Handling

The system is designed to keep running smoothly when individual parts fail. The table below summarises the main failure scenarios and how each is handled.

Failure scenario	What happens	Effect on the rest of the system
Sensor disconnects cleanly	The server closes that sensor's task and logs the event.	No effect — other sensors continue normally.
Sensor drops off unexpectedly	The server detects the broken connection and closes the task. The sensor waits, then tries to reconnect, backing off gradually if the server is not immediately available.	No effect — the slot is freed immediately.
Sensor sends a garbled message	The server logs a warning and skips the bad message. If five consecutive messages from the same sensor all fail to parse, the connection is closed.	No effect on other sensors or clients.
Storage write fails	The error is logged. The reading is still broadcast to live dashboard clients so real-time monitoring is unaffected.	Historical record for that reading is lost.
Storage read fails (REST query)	The API returns a 503 Service Unavailable response to the client.	Live dashboard is unaffected.
WebSocket client disconnects	The server removes that client from the broadcast list immediately.	No effect — remaining clients continue receiving readings.
WebSocket client is too slow	The server waits up to 2 seconds, then disconnects the client and removes it from the list.	Other clients are unaffected.
Server is overloaded	Because AsyncIO tasks are lightweight, the server can handle hundreds of sensors and clients simultaneously. Under extreme load the first bottleneck would be SQLite write throughput.	Readings may be batched to reduce write frequency.